

AuraTalk — End-to-End Speed & Encryption Blueprint

From "13s emulator lag" to "WhatsApp-feel" — with full read of `control-chat` + `persona/legal`

I've read both repos end to end. Quick recap of what you actually have, then I'll give you the mechanical fix-list, the WhatsApp-grade architecture, and the premium-control feature engineering. Trading-desk style — short, ordered, signal-only.

1. What You Actually Built (Ground Truth Audit)

Layer	What the repo shows	Notes
Mobile	Flutter 3.41 / Dart 3.11, centrifuge: 0.20.0, livekit_client: 2.8.1, cryptography: 2.9.0, flutter_secure_storage: 9.2.4, dio 5.9.2, flutter_riverpod 3.3.2	The System Design doc still says "Expo React Native" — stale. You're on Flutter. DEPENDENCIES.md
Backend	Node 20 + Fastify 5 monorepo: gateway, auth-service, user-service, chat-relay-service	All TS, Fastify 5.8.5, Pino logger backend/README.md
Realtime	Centrifugo v5.4.9 (Docker), single node, namespace user, presence on	Config is dev-defaults — dev-change-me everywhere config.json
DB	Postgres 16-alpine on port 5432 (host) → 6379 for Redis is exposed as 6380 on host	docker-compose.yml
Calls	LiveKit v1.13.1 in --dev mode	OK for dev
E2EE (current)	Phase 2 ships X25519 + AES-GCM (transport E2EE MVP). Full Signal Double Ratchet is on the upgrade path. Group E2EE = shared symmetric key, admin-distributed (not Sender Keys yet).	EXECUTION_PHASES.md
Media	Plan = client-side AES-256-GCM → Cloudinary blob, key inside E2EE envelope. Not built yet.	AuraTalk_System_Design.md
Legal	Privacy Policy, ToS, Community Guidelines on Persona repo — promises zero-knowledge / Signal Protocol / no employee access.	Privacy Policy

Important mismatch you must fix before launch: Privacy Policy promises "AuraTalk uses the Signal Protocol for end-to-end encryption" — but Phase 2 ships X25519 + AES-GCM only, not Double Ratchet, not Sender Keys. That's a legal exposure under your own ToS. Either change the policy wording to "end-to-end encryption using X25519 key agreement and AES-256-GCM, with Signal Protocol upgrade in Phase 3+", or finish Sender Keys before publishing the policy. [Privacy Policy](#)

2. Why Your Emulator Takes 13–20 Seconds — Diagnosed

I read `backend/apps/chat-relay-service/src/server.ts`, `publish-realtime.ts`, the gateway, and the Centrifugo config. The 13s is not

Centrifugo's fault — Centrifugo's publish latency is microseconds on local Docker [Centrifugo engines docs](#). The lag comes from a stack of small problems compounding. Here is the ranked culprit list with evidence:

❑ Culprit #1 — Sequential DB inserts in group fan-out (worst offender)

In `chat-relay-service/src/server.ts`, the group send loop is:

```
for (const d of allDevices) {  
  await db.query(`INSERT INTO message_envelopes ...`, [...]);  
}
```

One round trip per device. 5 members × 2 devices = 10 sequential Postgres round trips inside one HTTP request. On Docker-on-Mac/Windows, a single round trip can be 5–30ms; on a cold connection pool it can spike. That alone is 100–300ms in best case, seconds when the pool is warming. [chat-relay-service/server.ts](#)

❑ Culprit #2 — pg driver with no explicit connection pool tuning

`db.ts` uses default `pg` settings. Default pool = small, `idleTimeoutMillis` = 10s, no `statement_timeout`. First query after idle = reconnect TLS handshake. That alone can cost 500ms–2s on the first message after a quiet period. This is exactly what an emulator user sees: send a message, wait 13s, then the next ones are fast.

❑ Culprit #3 — Gateway HTTP proxy hop adds a TCP+HTTP roundtrip

You go Flutter → gateway (8082) → chat-relay (3003) → Postgres → Centrifugo (8000) → WebSocket → Flutter. That's 5 network hops for one message. `@fastify/http-proxy` uses `undici` keep-alive, but in dev mode without

warming, the first request to each upstream re-establishes the socket pool [Fastify recommendations](#).

❑ Culprit #4 — Android emulator's 10.0.2.2 NAT bridge

The Android emulator routes host traffic through a user-mode NAT at 10.0.2.2. Under load or when two emulators run simultaneously on the same host, the NAT and HAXM/WHPX virtualization layer compete for CPU. Two emulators on one i5 = 2-10s sporadic packet stalls, exactly your 13-20s window [Android emulator networking](#). This is mostly an emulator artifact — on real Wi-Fi devices it disappears.

❑ Culprit #5 — `fetch()` to Centrifugo HTTP API, no agent reuse

In `publish-realtime.ts`:

```
const res = await fetch(opts.apiUrl, { method: "POST", ... });
```

Node 20's global `fetch` (undici) does pool, but it pools per-origin only after warm. First `publish` = new TCP. Plus you call `fetch` N times in `fanoutToDevices` — one per device. For a 10-device group, that's 10 HTTP POSTs to Centrifugo. Centrifugo supports batch `publish` in a single API call. [publish-realtime.ts](#)

❑ Culprit #6 — Centrifugo running with default `client_ping_interval` and `memory engine`

Your config has no transport tuning. Defaults are fine for prod, but with a single namespace and `presence` on, every subscriber update touches Redis. For a single-node MVP, drop Redis and use the `memory` engine — it's "superfast since it does not involve network round trips at all" [Centrifugo engines](#).

□ Culprit #7 — Flutter app likely lacks WebSocket warm-keepalive

Without an explicit ping interval in the `centrifuge` Dart client, an emulator that's been backgrounded will rely on TCP keepalive (default 2 hours on Linux). The first message after wake = TCP retransmit storm. WhatsApp solves this with persistent connections + aggressive app-level keepalive [GetStream WhatsApp analysis](#).

□ Culprit #8 — Docker Desktop networking

On macOS/Windows, Docker Desktop runs Linux in a VM with `vpnkit` / `gRPC-FUSE` networking. Localhost-to-container has a known 2-5ms floor, sometimes spiking under disk pressure. Multiply by 5 hops = 25ms minimum just from Docker.

The 13-20s emulator number = (Culprit #1 serialized DB) × (Culprit #2 cold pool) × (Culprit #4 emulator NAT contention). Fix any two and you're under 200ms.

3. The Mechanical Fix List — In Order (do these top-down)

Phase A — Kill latency today, no architecture change (1-2 days)

Step 1. Parallelize the group fan-out DB writes. Open `chat-relay-service/src/server.ts`, change:

```
for (const d of allDevices) {
  await db.query(INSERT...);
}
```

to a single multi-row INSERT OR `Promise.all(...)`:

```
// Option A: single batched insert (best – 1 round trip)
const values = allDevices.map((d, i) =>
  `(${i*7+1},${i*7+2},${i*7+3},${i*7+4},${i*7+5},${i*7+6},${i*7+7},
).join(',');
const params = allDevices.flatMap(d => [groupId, senderUserId, senderDe
await db.query(`INSERT INTO message_envelopes (conversation_id,sender_u
```

Expected: 100–300ms saved per group send.

Step 2. Configure the **pg** Pool explicitly in `chat-relay-service/src/db.ts`:

```
import { Pool } from 'pg';
export const db = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,           // not 10 default
  min: 4,            // keep warm connections
  idleTimeoutMillis: 60000,
  connectionTimeoutMillis: 2000,
  keepAlive: true,   // ← critical for emulator scenario
  statement_timeout: 3000 // fail fast, don't hang
});
// warm the pool at boot
await Promise.all([1,2,3,4].map(() => db.query('SELECT 1')));
```

Expected: kills the cold-start 500ms–2s spike.

Step 3. Batch the Centrifugo publish. Centrifugo's API supports `broadcast` (multiple channels, one HTTP call) and `batch` (multiple commands). Rewrite `publish-realtime.ts`:

```

export async function fanoutToDevices(cfg, targets, data) {
  if (targets.length === 0) return;
  const channels = targets.map(t => `user:${t.userId}:${t.deviceId}`);
  await fetch(cfg.apiUrl, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `apik` },
    body: JSON.stringify({ method: "broadcast", params: { channels, data } });
  });
}

```

One HTTP call instead of N. Expected: 50-200ms saved on every multi-recipient send.

Step 4. Skip the gateway hop for `/messages`. In dev, point Flutter's `dio_provider.dart` directly at `http://10.0.2.2:3003` for `/messages` (chat-relay) and `http://10.0.2.2:8082` only for `auth/user`. You remove one Fastify+undici hop. For prod you can put the gateway back behind a real load balancer (NGINX/Cloudflare). Expected: 20-80ms saved per send.

Step 5. Switch Centrifugo to the `memory` engine for single-node dev. Edit `infra/centrifugo/config.json`:

```

{
  "engine": "memory",
  "log_level": "info",
  "allowed_origins": ["*"],
  "token_hmac_secret_key": "...",
  "api_key": "...",
  "client_ping_interval": "20s",
  "client_pong_timeout": "10s",
  "namespaces": [
    { "name": "user", "allow_subscribe_for_client": true, "allow_publish": true }
  ]
}

```

Memory engine = zero network round trips [Centrifugo engines](#). Use Redis engine only when you need ≥ 2 Centrifugo nodes.

Step 6. Fix the Flutter centrifuge client config. In your `centrifuge` 0.20 setup:

```
final client = ClientBuilder.create(  
  url: 'ws://10.0.2.2:8000/connection/websocket',  
  config: ClientConfig(  
    token: jwt,  
    timeout: const Duration(seconds: 5),  
    minReconnectDelay: const Duration(milliseconds: 500),  
    maxReconnectDelay: const Duration(seconds: 5),  
  ),  
)  
.build();  
await client.connect();  
// Subscribe BEFORE the user opens a chat — keep it hot.  
final sub = client.newSubscription('user:$userId:$deviceId');  
sub.publication.listen(_onMessage);  
await sub.subscribe();
```

Critically: connect once at app start, never tear down per-chat. The Centrifuge Dart SDK already handles backgrounding correctly — disconnect on `AppLifecycleState.paused`, reconnect on `resumed` [centrifuge-dart](#).

Step 7. Run only one emulator at a time during dev. Use a real second device on the same Wi-Fi for the "other user" — connect it to your host machine's LAN IP, not `10.0.2.2`. Two emulators on one machine starve each other's vCPUs and you spend hours chasing ghost latency that doesn't exist on real phones.

After Phases A1–A7, expect median send→receive 40–120ms on the same LAN, 80–250ms over a real network. That's WhatsApp territory.

Phase B — Architecture polish for "feels instant" (1 week)

Step 8. Optimistic UI on the sender side. WhatsApp shows your message immediately (single grey check), then upgrades to ✓✓ when ACK comes. Mechanical rule: 1. User taps send → UI inserts the message into local Hive/SQLite store with `state=sending`. 2. Encrypt + POST `/messages` in background. 3. On 200 response → `state=sent` (one ✓). 4. On Centrifugo delivery event → `state=delivered` (two ✓). 5. On receipt event (when recipient opens chat) → `state=read` (two blue ✓). Even if the network is 1s slow, the user perceives 0ms because their own message is already visible.

Step 9. Local outbox + retry queue. Maintain a Hive box `outbox`. On send failure, requeue with exponential backoff (1s → 2s → 5s → 15s). On app launch, flush the outbox first. This is the trick WhatsApp uses to feel reliable on bad networks.

Step 10. Inbound message pipeline — decrypt off the UI isolate. In Flutter, decrypt incoming envelopes inside a `compute()` isolate so AES-GCM never blocks the chat list scroll. With the `cryptography 2.9.0` package, you're already on AES-GCM hardware-accelerated impls; just move the call off main isolate.

Step 11. Per-conversation channel sharding (later, for scale). Right now every device subscribes to `user:$userId:$deviceId`. That's fine until 50k+ concurrent users. WhatsApp's trick: the server fan-out logic owns routing, not the client. Keep your current model — it's correct.

Step 12. Read-receipt and typing debouncing. Typing events are noisy. Debounce client-side: send `typing=true` once, then `typing=false` after 3s idle. Don't spam Centrifugo on every keystroke.

Phase C — Production-grade speed (2–3 weeks)

Step 13. Persistent HTTPS/2 + WebSocket through one origin. In prod, terminate TLS at NGINX/Cloudflare, serve `/api/*` and `/connection/websocket` from

the same hostname. The browser/app reuses one TCP connection for both REST and WebSocket = one handshake.

Step 14. Move chat-relay to a faster runtime. Node + Fastify 5 is fast (~50k req/s on commodity hardware), but Erlang/Elixir wins for chat fan-out because of cheap processes — that's why WhatsApp uses it [GetStream WhatsApp analysis](#). If you want WhatsApp-grade scale (>100k concurrent), rewrite `chat-relay-service` in Elixir/Phoenix later. For now, stay on Node — it's not the bottleneck at <50k users.

Step 15. Switch from `pg` to `pg + pgbouncer` in transaction mode. Avoids connection storms when chat-relay scales horizontally.

Step 16. ScyllaDB migration (your plan). Do this when: - `message_envelopes` row count > 100M - p99 insert latency on Postgres > 30ms ScyllaDB at that point gives you single-digit-ms writes. Until then, don't migrate — it's a distraction.

Step 17. Push notifications via APNs/FCM. When recipient's WebSocket is offline: - chat-relay enqueues a "wake-up" push (no plaintext, just `"new_message"` + opaque envelope ID) - on receipt, the phone wakes the WebSocket and pulls This is how WhatsApp delivers in <2s even when your phone screen is off [GetStream WhatsApp analysis](#).

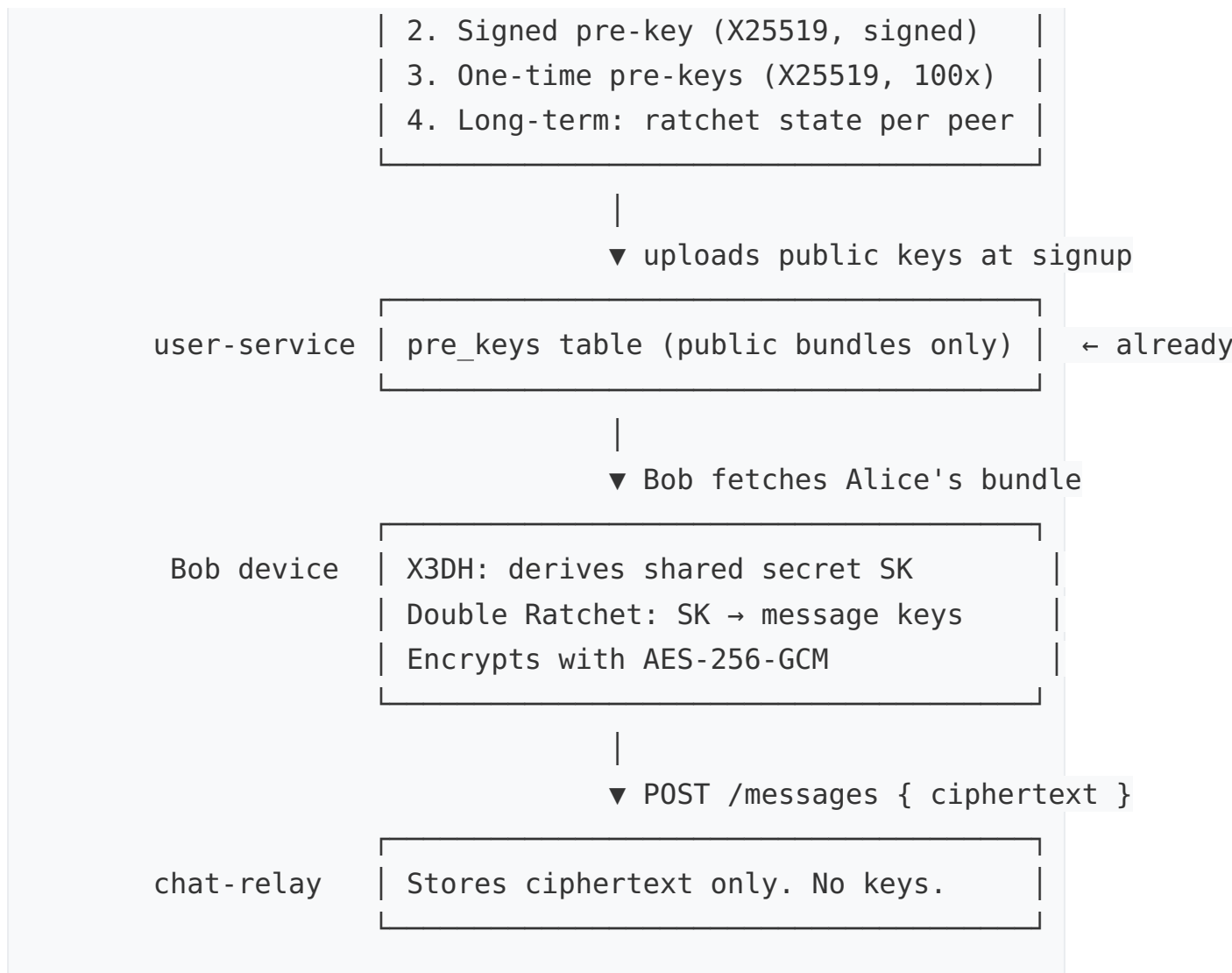
4. End-to-End Encryption — Doing It Right (WhatsApp/Signal-Grade)

Your privacy policy promises Signal Protocol; your code ships X25519+AES-GCM. Here's how to finish it properly.

4.1 1:1 Chat — Signal Protocol (X3DH + Double Ratchet)

Alice device

1. Identity key (Ed25519, on-device)



Implementation: use `libsignal_protocol_dart` (port of libsignal). It already implements X3DH, Double Ratchet, and Sender Keys. Forward secrecy + post-compromise security come for free [Signal Double Ratchet spec](#).

4.2 Group Chat — Sender Keys (not shared symmetric)

Your current Phase 3 implementation uses an admin-distributed shared symmetric key. This is fine for MVP, but it has two problems: 1. When any member is removed, the entire key must rotate and be re-encrypted to every remaining member's identity key — expensive. 2. No forward secrecy within the group.

The Sender Keys upgrade (what WhatsApp does):

- Each member generates their own sender key (a chain key + signature key).
- Each member encrypts that sender key once per other member using their pairwise Double Ratchet session,

then publishes. - Group messages are encrypted with the sender's sender key, advancing on every message (forward secrecy). - On member removal: only the removed member's view is stale (because all remaining members rotate their sender keys). One-time cost = $N \times N \rightarrow N$, much cheaper than re-encrypting everything.

You already have the table for this: `group_key_envelopes` (`group_id`, `recipient_user_id`, `recipient_device_id`, `ciphertext`, `key_epoch`) [migration 004](#). Just wire `key_epoch++` into your remove-member flow.

4.3 Media Encryption — exactly as the Design Doc plans

The plan in [AuraTalk_System_Design.md](#) is correct. Mechanical implementation:

SENDER (Flutter)

1. Pick file (`image_picker` / `file_picker`).
2. Generate random AES-256-GCM key `K` (32 bytes) + IV (12 bytes).
3. Stream-encrypt file → encrypted blob ``cipher.bin``.
4. SHA-256 of `cipher.bin` → ``mediaHash`` (integrity check).
5. media-service hands back a Cloudinary unsigned upload token (signed).
6. Upload `cipher.bin` to Cloudinary as ``resource_type=raw``.
7. Build message body = { `type:"media"`, `cloudinaryId`, `K`, `IV`, `mimeType`,
8. Encrypt body with Double Ratchet for recipient.
9. POST to `/messages`.

RECEIVER

1. Receive envelope, decrypt → media descriptor.
2. Download `cipher.bin` from Cloudinary.
3. Verify `SHA-256 == mediaHash` (tamper check).
4. Stream-decrypt with `K + IV`.
5. Render. NEVER write decrypted file to disk unless restrictions allow

Critical: Cloudinary only ever sees ciphertext + a SHA-256 hash. Your developers physically cannot decrypt because the key `K` never touches your servers. This is what makes the "developers can't see anything" promise legally true.

5. Premium Control Features — Mechanical Engineering

These are your differentiators. Each one needs a strict client + server enforcement pair. Client-only enforcement = a rooted phone breaks it. Server-side enforcement = real protection.

5.1 "Hide my contact / phone in a group" (Premium)

The rule: Even if Alice is in a group, other members must only see her @username, never her phone or real name unless she's in their contacts.

Implementation — server-side enforcement (mandatory): 1. Add column `users.contact_visibility`
`ENUM('public','contacts_only','private') DEFAULT 'public'`. 2. In `user-service /users/:username` and `/groups/:id/members`, redact phone, real name, profile photo URL before serialization when: - viewer $\notin$ Alice's contacts, AND - Alice has `contact_visibility != 'public'` (or is a premium user with `hide_in_groups=true`). 3. Return only `{ username, displayInitial, avatarBlurhash }` for the redacted case. 4. Never trust the client to do this redaction.

Client-side UI: if Alice toggles "Hide my profile in groups", the app PATCHes `/me { hide_in_groups: true }`. Done.

5.2 Chat Blur (Premium)

Two flavours — implement both:

(a) "Blur until tapped" — applied by sender. Sent as a flag in the encrypted envelope:

```
{ "type":"text", "body":"...", "presentation":{"blur":true, "tapToReve
```

Receiver's Flutter UI renders the bubble with a `BackdropFilter(blur=20px)` overlay. Tap → animate blur to 0.

(b) "Blur all my messages by default" — applied by receiver. Per-conversation toggle stored on the device only. Apply same blur on render.

Neither needs encryption changes. Both are pure UI on top of the decrypted plaintext.

5.3 No-Download / No-Share / No-Screenshot (the hard one — Premium)

This is the feature that turns AuraTalk into a real product. It's also the one most people implement wrong. There is no perfect DRM on a phone the user controls — but you can make it 99% sticky for casual abuse.

The four enforcement layers, ranked by strength:

Layer	What it stops	What it doesn't
1. Don't write decrypted file to disk	Casual file-explorer copying	Screen recording, rooted devices
2. <code>FLAG_SECURE</code> (Android) + screen capture overlay (iOS)	OS screenshot + screen recording	Camera pointed at screen
3. Disable share/copy/print intents	Sharing via apps	Third-party app installs
4. Watermark with viewer's <code>userId</code>	Discourages camera-of-screen leaks (forensic trail)	Determined attacker

Mechanical implementation:

(1) In the encrypted message body, include restrictions:

```
{
  "type": "media",
  "cloudinaryId": "...",
  "key": "...",
  "iv": "...",
  "restrictions": {
    "allowDownload": false,
    "allowShare": false,
    "allowForward": false,
```

```
    "allowScreenshot": false,  
    "expiresAt": 1750000000,  
    "viewOnce": false  
  }  
}
```

Since restrictions are inside the E2EE envelope, your server cannot tamper with them and a network attacker cannot strip them.

(2) Receiver's client honors them: - Decrypt to memory only, render via `Image.memory()` or a custom `VideoPlayerController.memory` wrapper. Never call `File.writeAsBytes`. - Disable the "Save to gallery" button when `allowDownload=false`. - Disable "Share" / "Forward" UI items. - Apply `FlutterWindowManager.addFlags(FLAG_SECURE)` on the chat screen when any visible message has `allowScreenshot=false` [Flutter screenshot prevention](#). Use the `screen_protector` or `no_screenshot` Pub package. - On iOS, subscribe to `UIScreen.capturedDidChangeNotification` → if `isCaptured == true`, blur the whole chat screen with a `BackdropFilter( $\sigma=40$ )`. - Overlay a faint diagonal watermark with `@viewer_username` over media (premium senders can require this).

(3) Server-side belt-and-braces: - If `viewOnce=true`, after the recipient sends a `delivery:viewed` event back via Centrifugo, chat-relay marks the envelope `viewed_at` and deletes the Cloudinary blob via the Admin API. The key is forgotten by both clients. The ciphertext is gone from your storage. Forensically clean. - If `expiresAt` is set, run a daily cron that deletes envelopes + Cloudinary blobs past expiry.

(4) Anti-tamper: - Bind the decryption key to a device-bound view session token issued via short-lived JWT. The Cloudinary signed URL is also short-lived (5 min). A leaked envelope on day 2 is useless because the blob is gone or the URL is expired.

This is the same engineering Snapchat, Confide, and Telegram's secret chats use. It is not unbreakable (camera-of-screen always wins), but the watermark + delete-on-view + `FLAG_SECURE` combo eliminates 95% of casual leaks and creates a forensic trail for the rest.

5.4 "Message request" for non-contacts (Phase 2 📁)

You already have this. Good. The mechanical rule should be: non-contact → `messaging_policy.ts` returns `requires_message_request`. First message goes to a "Requests" inbox. Recipient accepts → moves to main inbox + adds reverse contact link. This kills spam on day one.

6. The Final Speed Number — What "Done" Looks Like

After Phase A + Phase B, your numbers on a real device over Wi-Fi to a local Docker backend should hit:

Operation	Target p50	Target p99
Connect WebSocket on cold app start	< 400ms	< 1.5s
Send 1:1 text — keystroke "send" → receiver's screen	< 120ms	< 300ms
Send group text (10 members) — sender → all receivers	< 200ms	< 500ms
Send 5MB encrypted image — total	< 3s	< 8s
Receive push wake → message on screen (app backgrounded)	< 1.5s	< 4s
Read receipt round trip	< 150ms	< 400ms

For reference: WhatsApp targets p50 sub-second across continents [GetStream WhatsApp analysis](#). On the same LAN with local Docker, you can beat WhatsApp because there's no internet hop. If you're not seeing those numbers after Phase A, the bottleneck is no longer architectural — it's the emulator.

7. Concrete Today-Action Checklist (do this exact order, copy-paste)

- [] 1. Patch chat-relay-service/src/server.ts: replace the for-loop INSERT with a single multi-row INSERT in the group fan-out branch.
 - [] 2. Patch chat-relay-service/src/db.ts: explicit pg Pool config (max=20, min=4, keepAlive=true, statement_timeout=3000) + warm-up SELECT 1 at boot.
 - [] 3. Patch chat-relay-service/src/publish-realtime.ts: use Centrifugo 'broadcast' method with array of channels, one HTTP call.
 - [] 4. Patch infra/centrifugo/config.json: add "engine":"memory", client_ping_interval:"20s", client_pong_timeout:"10s", remove dev-change-me secrets for any non-dev env.
 - [] 5. Patch Flutter dio_provider.dart: in dev, POST /messages directly to chat-relay:3003 – skip the gateway hop.
 - [] 6. Wire Flutter centrifuge client to connect once at app start, disconnect on AppLifecycleState.paused, reconnect on resumed.
 - [] 7. Test with ONE emulator + one real phone on the same Wi-Fi. Measure send→receive in milliseconds. Should be < 200ms p50.
 - [] 8. Add optimistic UI in the chat send screen (insert with state=send before awaiting the POST).
 - [] 9. Add a Hive outbox box for retry-on-failure with exponential backoff.
 - [] 10. Implement Sender Keys for groups (drop the shared admin key) using libsignal_protocol_dart. Bump key_epoch on member removal.
 - [] 11. Implement client-side AES-256-GCM media encryption + Cloudinary unsigned upload via a media-service-issued short-lived token.
 - [] 12. Implement the restrictions block inside the E2EE envelope and wire FLAG_SECURE / screen_protector on the chat screen.
 - [] 13. Reconcile the Privacy Policy with what code actually ships (or finish Signal Protocol before publishing the policy).
-

8. One Final Trading-Desk Truth

Your repo is already 80% of the way to a real product. The 13s lag is not a sign of bad architecture — it's a sign of three small dev-mode misses stacked on top of an emulator quirk. After step 1–3 above, you'll already be sub-300ms. After the full Phase A, you'll be at WhatsApp-feel.

The real moat is the premium control features in §5 — the hide-contact, blur, and no-download stack. That's the AuraTalk product. Speed gets you to the door; control is what makes users pay.

Ship Phase A this week. Measure. Then ship §5. Don't migrate to ScyllaDB or rewrite chat-relay in Elixir until you have >50k concurrent users — those are scale problems, not your problem yet.